

*b KB

IT's Your Life

with





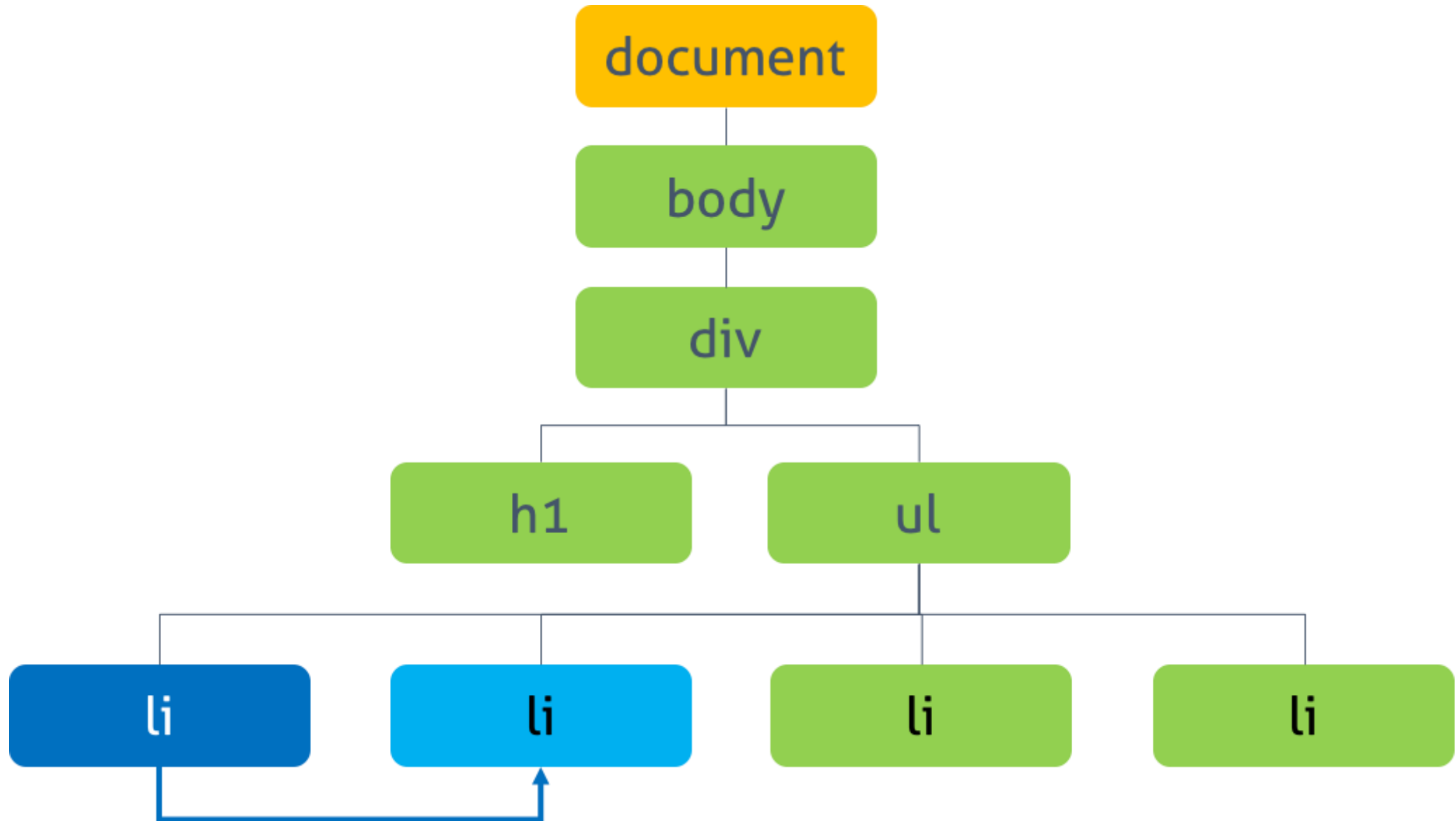
DOM

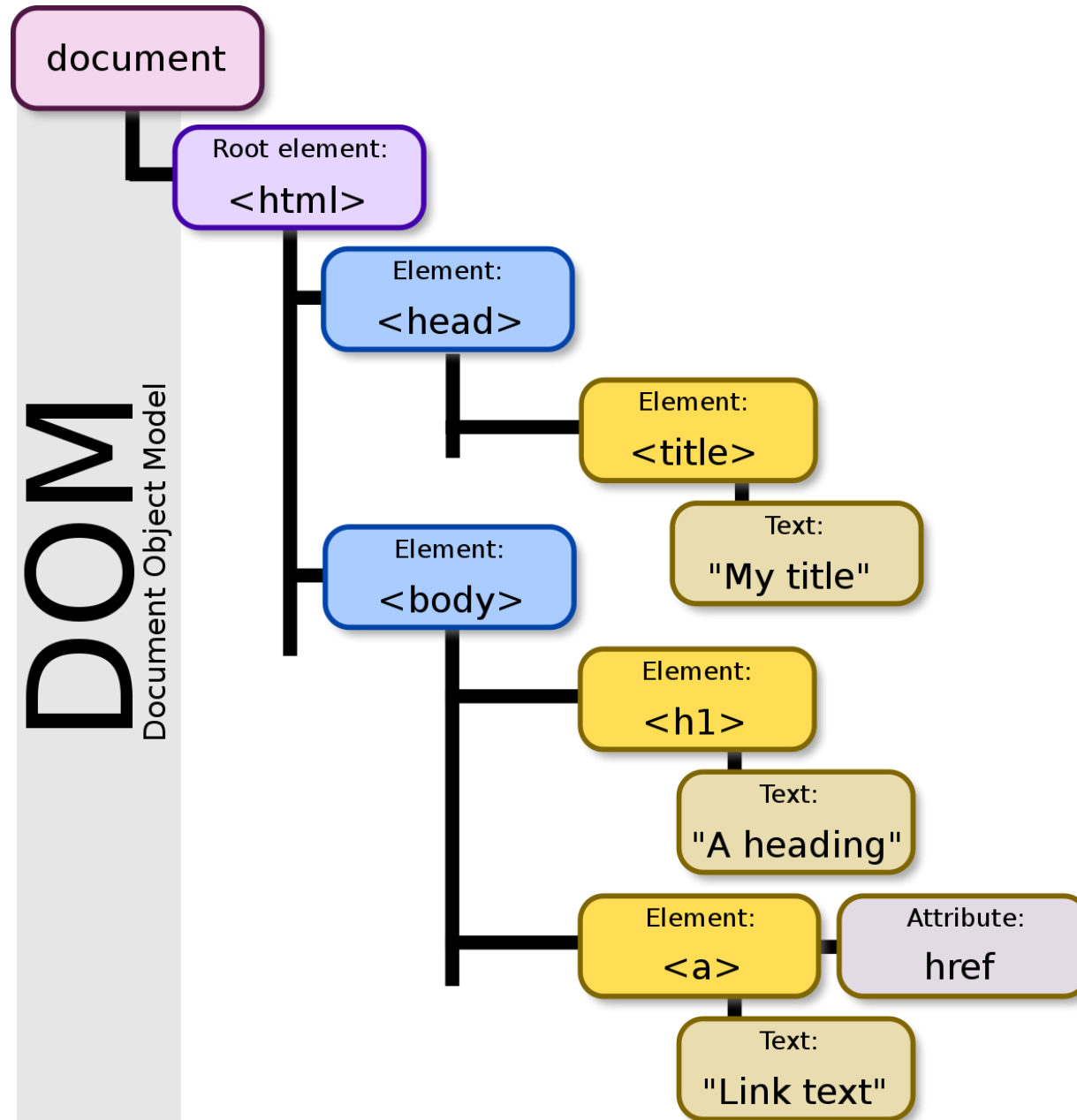
(Document Object Model)

DOM(Document Object Model)



- HTML 문서 요소의 집합!
- HTML 문서는 각각의 node 와 object 의 집합으로 문서를 표현
- 따라서 각각 node 또는 object 에 접근하여 문서 구조 / 스타일 / 내용 등을 변경 할 수 있도록 하는 것!







DOM API

Document Object Model, Application Programming Interface



```
// HTML 요소(Element) 1개 검색/찾기
const boxEl = document.querySelector('.box');

// HTML 요소에 적용할 수 있는 메소드!
boxEl.addEventListener();

// 인수(Arguments)를 추가 가능!
boxEl.addEventListener(1, 2);

// 1 - 이벤트(Event, 상황)
boxEl.addEventListener('click', 2);

// 2 - 핸들러(Handler, 실행할 함수)
boxEl.addEventListener('click', function () {
    console.log('Click~!');
});
```



결국 JS 가 HTML 요소를 선택하고
해당 요소에게 동적 작업을
시키는 관계라고 생각하시면
편합니다~!





이러한 관계는 CSS 와 비슷하죠?

당연하게 JS 도 HTML 을
정확하게 선택해야만 합니다!



HTML 요소를 선택하는 방법들



querySelector

querySelector("요소 선택자")



- CSS 에서 배웠던 선택자를 사용하여 HTML 을 선택하는 방법 입니다
- 문서에서 만나는 **제일 첫번째 요소**를 반환 합니다!

```
<body>
  <div class="box">class</div>
  <script>
    const box = document.querySelector('.box');
    console.log(box);
  </script>
</body>
```

```
<div class="box">class</div>
```



getElementById

getElementById("ID")



- 해당 ID를 가지는 요소를 불러오는 메서드

```
<body>
  <div class="box">class</div>
  <div id="box">id</div>
  <script>
    const box = document.getElementById('box');
    console.log('🚀 ~ box:', box);
  </script>
</body>
```

```
🚀 ~ box: ▶ div#box
```



querySelectorAll

querySelectorAll("요소 선택자")



- 선택자와 일치하는 모든 요소를 찾아 배열로 리턴해주는 메서드

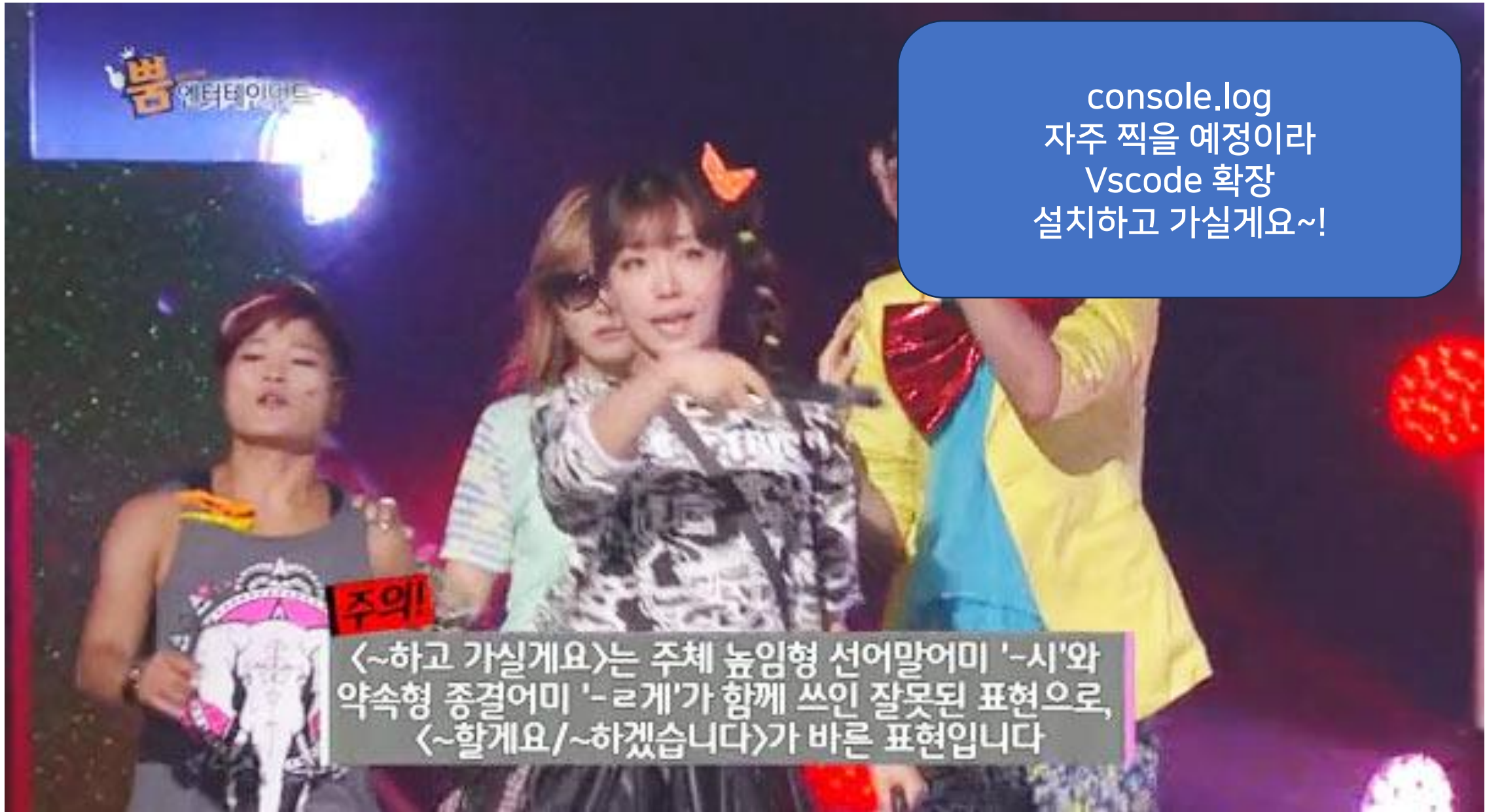
```
<body>
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <script>
    const boxEls = document.querySelectorAll('div');
    console.log('🚀 ~ boxEls:', boxEls);
  </script>
</body>
```

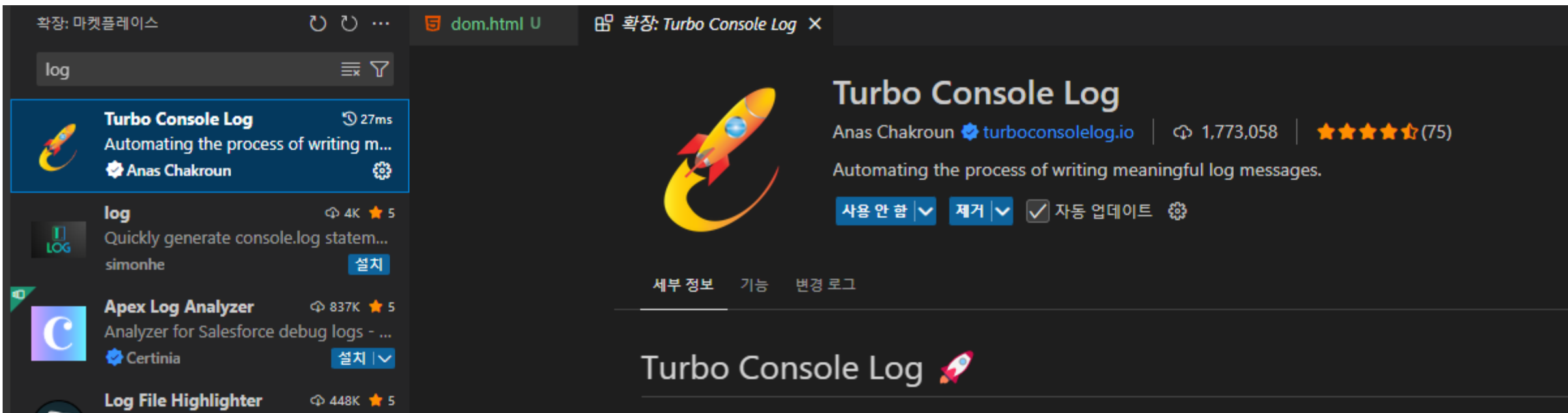
```
🚀 ~ boxEls:
▼ NodeList(7) ⓘ
  ▶ 0: div
  ▶ 1: div
  ▶ 2: div
  ▶ 3: div
  ▶ 4: div
  ▶ 5: div
  ▶ 6: div
  length: 7
  ▶ [[Prototype]]: NodeList
```






console.log
자주 찍을 예정이라
Vscode 확장
설치하고 가실게요~!





콘솔에 찍고 싶은 변수에 가서
Ctrl + Alt + L (windows) / Ctrl + Option + L (Mac)
누르면 알아서 console.log 를 자동 생성 해줍니다!



onclick!

onclick



- 각각의 HTML 요소에 속성 값으로 JS 함수를 연결

```
<body>
  <div class="box" onclick="test();">click</div>
</body>
```

```
function test() {
  alert("TEST");
}
```

이 페이지 내용:

TEST

확인



addEventListener

addEventListener(이벤트, 명령)



- 선택 요소에 지정한 이벤트가 발생하면, 약속 된 명령어를 실행시키는 메소드



```
<head>
  <style>
    .box {
      width: 100px;
      height: 100px;
      background-color: skyblue;
    }
  </style>
</head>
<body>
  <div class="box">1</div>
  <script>
    const boxEl = document.querySelector('.box');

    function alertFunc() {
      alert('click');
    }
    console.log(boxEl);

    boxEl.addEventListener('click', alertFunc);
  </script>
</body>
```




addEventListener 이벤트의 종류

- Click : 클릭
- Mouse 계열
 - Mouseover : 요소에 커서를 올렸을 때
 - Mouseout : 마우스가 요소를 벗어날 때
 - Mousedown : 마우스 버튼을 누르고 있는 상태
 - Mouseup : 마우스 버튼을 떼는 순간
- Focus : 포커스가 갔을 때
- Blur : 포커스가 벗어나는 순간

addEventListener 이벤트의 종류



- Key 계열
 - Keypress : 키를 누르는 순간 + 누르고 있는 동안 계속 발생
 - Keydown : 키를 누르는 순간에만 발생
 - Keyup : 키를 눌렀다가 떼는 순간
- Load : 웹페이지에 필요한 모든 파일(html, css, js 등)의 다운로드가 완료 되었을 때
- Resize : 브라우저 창의 크기가 변경 될 때
- Scroll : 스크롤이 발생할 때
- Unload : 링크를 타고 이동하거나, 브라우저를 닫을 때
- Change : 폼 필드의 상태가 변경 되었을 때

실습, 버튼을 클릭하면 오늘의 날짜를 보여주기

- 아래의 코드를 활용해서 버튼을 클릭하면 오늘의 날짜를 console.log 로 출력하는 프로그램을 완성하세요
- 단, addEventListener 를 활용하여 구현해 주세요
- 오늘 날짜는 Date 객체를 그대로 출력해도 됩니다!



```
<body>  
  <button id="dateBtn">오늘은?</button>  
  <script>  
    const btnEl = document.getElementById('dateBtn');  
  </script>  
</body>
```

Tue Mar 11 2025 11:43:46 GMT+0900 (한국 표준시)



this 와
e.target

this 활용하기



- DOM 요소에서 this 를 사용하면, this 는 자기 자신 Node 를 가르킵니다!
→ onclick 에서 주로 사용!

```
<ul>
  <li onclick="showThis(this);">1</li>
  <li onclick="showThis(this);">2</li>
  <li onclick="showThis(this);">3</li>
  <li onclick="showThis(this);">4</li>
</ul>
```

```
function showThis(t) {
  console.log(t);
}
```

```
▶ <li onclick="showThis(this);">...</li>
```

e.target 활용하기



- Document 에서 발생하는 이벤트는 이벤트 객체 e 를 통해 확인 할 수 있습니다. → addEventListener 에서 주로 사용!

```
<ul>
  <li class="list">1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
const list = document.querySelector(".list");

list.addEventListener("click", function (e) {
  console.log(e.target);
});
```

```
▶ <li class="list">...</li>
```



classList

classList.add / remove / contain



- 선택 요소에 class 를 더하거나, 빼거나, 클래스가 존재하는지 체크하는 메소드
- 해당 기능과 CSS 를 잘 활용하면 변화무쌍(?)한 웹페이지 구성이 가능

```
<head>
  <style>
    .box {
      width: 100px;
      height: 100px;
      border: 1px solid black;
    }
    .orange {
      background-color: orange;
    }
  </style>
</head>
<body>
  <div class="box"></div>
  <script>
    const boxEl = document.querySelector('.box');
    boxEl.addEventListener('click', function () {
      if (boxEl.classList.contains('orange')) {
        boxEl.classList.remove('orange');
      } else {
        boxEl.classList.add('orange');
      }
    });
  </script>
</body>
```





setAttribute

setAttribute, html 요소 속성 추가



- 선택한 요소의 속성 값을 직접 지정할 수 있는 메소드
- 요소.setAttribute("속성명", "지정할 속성")

```
searchInputEl.setAttribute("placeholder", "통합검색");
```

```
boxEl.setAttribute("style", "background-color: orange");
```



textContent



```
<body>
  <div class="box"></div>
  <script>
    let boxEl = document.querySelector('.box');
    console.log(boxEl.textContent);

    boxEl.textContent = `KB It's your Life`;
    console.log(boxEl.textContent);
  </script>
</body>
```

KB It's your Life

[lecture.html:12](#)

[lecture.html:14](#)



createElement

createElement, html 요소 생성



- Html DOM 요소를 만들어내는 메소드
- document.createElement("요소 이름")

```
const li = document.createElement("li");
```

```
const box = document.createElement("div");
```




append,
appendChild

append / appendChild, 요소 붙이기



- 특정 DOM 요소에 다른 요소를 자식으로 붙이는 메소드
- DOM요소.append(추가할 내용)
- DOM요소.appendChild(추가할 요소)

```
const li = document.createElement("li");
const checkBtn = document.createElement("input");
checkBtn.setAttribute("type", "checkbox");
li.append(checkBtn);
```

append()



- append() 는 Node 와 String 을 전부 추가 할 수 있어요
- append() 는 여러 가지 값을 한번에 붙일 수 있어요
- append() 는 반환(리턴) 값이 없어요!

appendChild()



- appendChild() 는 Node 만 추가할 수 있어요
- appendChild() 는 한 번에 하나만 추가할 수 있어요
- appendChild() 는 추가한 Node 를 반환(리턴) 합니다!



remove

remove, 선택한 DOM 을 지우는 메소드



- DOM요소.remove

```
<ul>
  <li>1</li>
  <li class="list">2</li>
  <li>3</li>
  <li>4</li>
</ul>
<script>
  const list = document.querySelector('.list');
  console.log(list);

  list.remove();
  console.log(list);
</script>
```

- 1
- 3
- 4

```
<li class="list">2</li>
<li class="list">2</li>
```

실습, 간단한 TodoList 만들어 보기



- 다음 강의 코드를 활용해서 아래의 조건을 만족하는 프로그램을 완성 하세요
- 할 일 추가 버튼을 클릭하면 “이 닙기” 가 li 의 자식 li 로 추가 됩니다
- 해당 li 에는 삭제 기능이 존재하며, 자기 자신을 클릭하면 li 가 삭제됩니다

```
<body>
  <button onclick="addTodo()">할 일 추가</button>
  <ul id="todolist"></ul>

  <script>
    const list = document.querySelector('#todolist');
  </script>
</body>
```



